Medical Triage Knowledge Graph

A Multi-Ontology GraphRAG Pipeline for Emergency Department Decision Support

LMP2392 — Healthcare and Medicine for AI Experts

March 22, 2026

Outline

1. Motivation
2. System Architecture
3. Knowledge Graph Construction
4. Guideline Audit Pipeline
5. Data Processing Pipeline
6. Diagnotix Web Application
7. Summary

The Clinical Problem

Emergency Department triage is time-critical

- Clinicians must rapidly map presenting symptoms to the right diagnostic test under cognitive and time pressure.
- Clinical guidelines (AHA/ACC, ACR, NICE, CTAS) encode evidence-based rules — but they are dispersed across long PDFs and multiple organisations.
- LLMs alone can *hallucinate* clinical pathways; they lack grounded, citable reasoning.

Our answer: a knowledge graph as the ground truth layer.

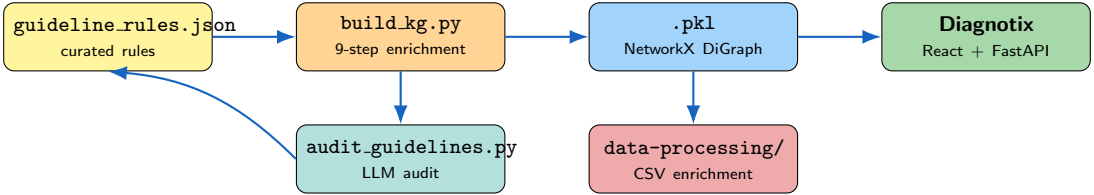
Target pathways

- ECG
- Testicular Ultrasound
- Arm X-Ray
- Abdominal Ultrasound
- Appendix Area Ultrasound

Guideline sources

AHA/ACC ACR NICE CTAS

High-Level Architecture



Rules are iteratively audited and revised until satisfactory, then `build_kg.py` serialises the final graph. The `.pkl` powers **Diagnotix** for dynamic expansion and `data-processing/` for patient CSV enrichment.

Source of Truth: guideline_rules.json

Each of the 45 rules encodes a single clinical assertion:

$$\texttt{raw_symptom} \longrightarrow \texttt{condition} \longrightarrow \texttt{test}$$

Example rule (JSON)

```
{
  "raw_symptom": "chest pain",
  "node_type": "Symptom",
  "condition": "Acute MI",
  "test": "ECG",
  "source": "AHA/ACC"
}
```

Rule fields

Field	Meaning
raw_symptom	Clinical entity
node_type	KG category
condition	Target diagnosis
test	Recommended test
source	Guideline body

9-Step Enrichment Pipeline (`build_kg.py`)

Step	API / Source	Enriches	Column(s) added
0	Static rules	—	base DataFrame
1	`GUIDELINE_URLS` dict	`(source, test)`	`url`
2	EMBL-EBI OLS4 (HP/MONDO/EFO)	`raw_symptom`	`ebi_code`, `synonyms`
3	Infoway FHIR (SNOMED CT CA)	`raw_symptom`	`infoway_code`
4	UMLS REST (LOINC)	`test`	`loinc_code`
5	UMLS REST (ICD-10-CM)	`condition`	`icd10_code`
6	NLM RxNorm + OpenFDA	`treatment`	`rxcui`, `adverse_event`
7	Europe PMC	symptom $\leftrightarrow$ condition	`literature_weight`
8	Europe PMC	condition $\leftrightarrow$ test	`test_literature_weight`
9	ClinicalTrials.gov v2	condition $\leftrightarrow$ treatment	`trial_count`

Steps 7–9 attach **real-world evidence counts** that drive edge-width scaling in the HTML visualisation.

Graph Schema

Primary node types

Symptom	`Symptom:`
Condition	`Condition:`
Test	`Test:`

Secondary node types

Vital	`Vital:`
Demographic	`Demographic:`
Risk Factor	`Risk Factor:`
MOI	`MOI:`
Attribute	`Attribute:`
Treatment	`Treatment:`

Edge types

`INDICATES_CONDITION`	entity → condition
`REQUIRES_TEST`	condition → test
`DIRECTLY_INDICATES_TEST`	entity → test
`RECOMMENDS_TREATMENT`	condition → drug

Edge metadata on every edge:

- `source` — guideline body (AHA/ACC, ACR, ...)
- `source_url` — citable DOI or page URL
- evidence weight (literature / trial count)

`audit_guidelines.py` — Three-Pass Audit

Problem: LLM-generated triage rules can be fabricated, vague, or misattributed. The audit pipeline retroactively cleans `guideline_rules.json` before it enters the graph.

Pass 1 — Schema & source pre-filter

Drops rules with missing required fields or unrecognised guideline bodies (allowlist of 33 organisations: AHA/ACC, ACR, NICE, CTAS, ESC, ...).

Pass 2 — LLM clinical verification (per pathway)

Rules are grouped by diagnostic test and sent to an Azure OpenAI reviewer. The LLM checks: (1) source accuracy, (2) clinical plausibility, (3) condition specificity. Only rules that pass all three are kept. *When in doubt, remove — false negatives are safer.*

Pass 3 — Name normalisation

`condition` → ICD-10-CM canonical description (NLM API). `raw_symptom` → HP/MONDO canonical label (EBI OLS4 API). Original string kept when no high-confidence match is found.

Data Processing Pipeline — Two-Phase Overview

Goal: ingest a real patient CSV and enrich every diagnosis with KG-derived diagnostic tests.

Phase 1 — Extract candidate tests

`csv_extract_potential_test.py`

Reads all unique diagnoses from the `dx` column and asks Azure OpenAI to identify the standard diagnostic test(s) for each. Writes `diagnoses_to_tests_graph_data.json`.

Phase 2 — Expand graph & map patients

`kg_enrichment_pipeline.py`

Adds any new test pathways to the KG, rebuilds the graph with full ontology enrichment, then maps every diagnosis back to a condition node and its recommended tests.

Standalone mapping (graph already current)

`csv_mapping.py` — skips graph expansion; semantically maps a new patient CSV against the existing PKL via Azure OpenAI.

Phase 1 — csv_extract_potential_test.py

Input: patient CSV with a dx (diagnosis) column.

Steps:

1. Collect all unique diagnosis strings from dx.
2. Batch them (default 20 per call) to Azure OpenAI.
3. LLM consolidates variant spellings into canonical test names suitable as KG node labels
   (e.g. all variants of "US Appendix" → "Appendix Ultrasound").
4. Write diagnoses_to_tests_graph_data.json.

Output (per diagnosis)

```
{
  "diagnosis":
    "Acute Appendicitis",
  "diagnostic_tests": [
    "Appendix Ultrasound",
    "Complete Blood Count"
  ]
}
```

Phase 2 — kg_enrichment_pipeline.py

Step	Action
1	Read `diagnoses_to_tests_graph_data.json`; collect unique test names.
1b	*(Optional)* LLM ranks candidates by clinical importance; keep top *N* via `--top-x`.
2	For each new test, call Azure OpenAI to generate 8–15 triage rules grounded in clinical guidelines.
3	Append new rules to `guideline_rules.json` (tests already present are skipped).
4	Rebuild the PKL via `build_kg.generate_knowledge_graph()` — full multi-ontology enrichment.
5	Semantically map every `dx` value to a graph condition node; write enriched CSV.

Checkpoint / resume

Step 5 writes a sidecar checkpoint after every LLM call. Interrupted runs resume from where they left off.

Output columns added to CSV

`matched_graph_condition`
best-matching `Condition:` node

`potential_tests`
tests linked via `REQUIRES_TEST`

Diagnotix — Dynamic Knowledge Graph Builder

Full-stack web application (FastAPI + React)

- Enter any diagnostic test name; the LLM extracts 8–15 evidence-based triage rules automatically.
- Generated rules are validated and audited via `audit_guidelines.py` before being committed to the graph.
- Validated rules are appended to `guideline_rules.json` and the full ontology enrichment pipeline re-runs.
- Every expansion is auditable: the response reports new rule count and updated node/edge totals.

POST /api/kg/expand

```
{ "diagnostic_test": "CT Head" }

-> {
  "new_rules_added": 11,
  "nodes": 98,
  "edges": 247
}
```

Stack

Backend: FastAPI + Uvicorn
Frontend: React 19 + TypeScript + Vite
LLM: Claude (Anthropic API)

Summary

What we built

- An expandable knowledge graph spanning 5 clinical pathways, grounded in evidence-based guidelines.
- 9-step ontology enrichment pipeline (SNOMED, LOINC, ICD-10, HP/MONDO, RxNorm, Europe PMC, ClinicalTrials.gov).
- LLM-powered audit pipeline to ensure rule accuracy and source attribution.
- Two-phase data processing pipeline for back-mapping patient diagnoses to recommended diagnostic tests.

Future work

- Continuously improve the audit pipeline to maintain knowledge graph accuracy over time.
- Integration with the SickKids database (where feasible).
- Extend from *back prediction* (diagnosis $\rightarrow$ test) to *front prediction* — using patient vitals at triage to recommend tests, grounded in the knowledge graph.
- Expert clinician verification of the constructed knowledge graph.